

Jagoz

Descrição da organização e instruções de execução do projeto

Aplicação web para apoio à gestão do GDUE: sócios, atletas, patrocinadores, eventos, bilhetes, pagamentos e ficheiros.

O repositório está dividido em dois blocos principais:

- `kotlin/project`: backend Kotlin + Spring Boot, organizado por módulos Gradle.
- `js`: frontend React + Vite.

A documentação da API está em `docs/openapi.yaml`.

Como aceder

1. Aceder ao repositório

<https://github.com/TomasPereira05/project>

2. Clonar o repositório

3. Realizar os passos em Configuração

Requisitos

- JDK 21
- Node.js 20 ou superior
- npm
- PostgreSQL 15 ou superior
- Docker, opcional, mas recomendado para testes Gradle com base de dados
- Conta/credenciais Stripe, SMTP e Cloudflare R2 para executar todos os fluxos reais

Configuração

1. Copiar o exemplo de ambiente:

```
Copy-Item .env.example .env
```

2. Preencher os valores do `.env`.

Nunca revelar as credenciais reais. O backend lê várias variáveis obrigatórias no arranque, incluindo `DB_URL`, credenciais SMTP, Stripe e R2.

3. Criar a base de dados PostgreSQL local:

```
CREATE DATABASE jagoz;
```

4. Aplicar o schema e, se quiser dados de desenvolvimento, os dados de teste:

```
psql "postgres://postgres:postgres@localhost:5432/jagoz" -f kotlin/project/modules/repository-jdbi/src/sql/JagozSchema.sql
```

```
psql "postgresql://postgres:postgres@localhost:5432/jagoz" -f kotlin/project/modules/repository-jdbi/src/sql/insert-test-data.sql
```

O DB_URL usado pela aplicação deve apontar para essa base de dados, por exemplo:

```
DB_URL=jdbc:postgresql://localhost:5432/jagoz?user=postgres&password=postgres
```

Como correr

Abrir dois terminais.

Terminal 1, backend:

```
cd kotlin/project
./gradlew.bat :host:bootRun
```

Por omissão o Spring Boot arranca em `http://localhost:8080`.

Terminal 2, frontend:

```
cd js
npm install
npm run dev
```

Abrir `http://localhost:5173`. O Vite faz proxy de `/api` para `http://localhost:8080`, configurado em `js/vite.config.ts`.

Como correr com Docker

Na raiz do repositório:

```
docker compose up --build
```

Servicos expostos:

- Frontend: `http://localhost:5173`
- Backend: `http://localhost:8080`
- PostgreSQL: `localhost:5432`, base `jagoz`, user `postgres`, password `postgres`

O Compose aplica automaticamente `JagozSchema.sql` e `insert-test-data.sql` quando o volume da base de dados é criado pela primeira vez. Para recriar a base do zero:

```
docker compose down -v
docker compose up --build
```

O backend recebe placeholders para Stripe, SMTP e R2 se essas variáveis não existirem no ambiente. Isto permite arrancar a app localmente, mas os fluxos reais de pagamentos, email e upload para R2 precisam de credenciais verdadeiras.

Contas de desenvolvimento

Quando `insert-test-data.sql` é aplicado, existem utilizadores de exemplo. A password dos utilizadores inseridos e a mesma hash usada no script de seed (`tomas123`);

Exemplos de usernames presentes no seed:

- tomas, role ADMIN
- tomas22, role SECRETARIA
- ana.costa, role NORMAL
- sponsor.atlantico, role NORMAL

Testes

Backend:

```
cd kotlin/project
./gradlew.bat test
```

Os testes dos módulos que precisam de PostgreSQL usam `kotlin/project/docker-compose.yml`, com o serviço `db-tests` na porta 5433.

Frontend:

```
cd js
npm run dev
```

Documentação adicional

- API: `docs/openapi.yaml`
- Backend: `kotlin/project/README.md`
- Frontend: `js/README.md`
- Relatório e recursos do projeto: `docs/`

Segurança

O ficheiro `.env` deve ficar apenas local. Se algum segredo real tiver sido partilhado ou commitado, deve ser trocado no fornecedor respetivo: Stripe, SMTP/email e Cloudflare R2.